

**SIMATS ENGINEERING**  
**Department of Computer Science and Engineering**  
**ASSESSMENT TOOL 2 - CONCEPT MAPPING**

---

|                       |                                                                                                                                                                                                                                                    |                      |                                             |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|---------------------------------------------|
| <b>Course Code:</b>   | CSA1205                                                                                                                                                                                                                                            | <b>Course Title:</b> | Computer Architecture for Multicore Systems |
| <b>CO Assessed:</b>   | CO1                                                                                                                                                                                                                                                | <b>Assessment:</b>   | ASSESSMENT TOOL 2- CONCEPT MAPPING          |
| <b>Weightage:</b>     | 35%                                                                                                                                                                                                                                                | <b>Total Marks:</b>  | 25                                          |
| <b>CO1 Statement:</b> | Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) |                      |                                             |
| <b>Student Name:</b>  | DEEPA PREYA H                                                                                                                                                                                                                                      | <b>Reg.No.:</b>      | 192421047                                   |
| <b>Department:</b>    | INFORMATION TECHNOLOGY                                                                                                                                                                                                                             | <b>Date:</b>         | 20.07.2026                                  |

**Instruction to Students**

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

**ANNEXURE A**

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
  2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
  3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
  4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
  5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.
-

**Code of Conduct:**

*I DEEPA PREYA H/ 192421047 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

**MARKS ALLOCATION**

| Criteria | Concept Identification (25%) | Linkages (25%) | Organization (20%) | Integration of Literature (20%) | Clarity (10%) |
|----------|------------------------------|----------------|--------------------|---------------------------------|---------------|
| Q1       |                              |                |                    |                                 |               |
| Q2       |                              |                |                    |                                 |               |

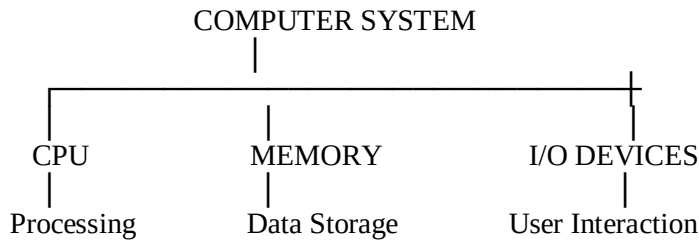
**RUBRICS FOR CALCULATION:**

| Criteria                  | Weightage | Level 4 – Exemplary                                                         | Level 3 – Proficient                              | Level 2 – Developing                                        | Level 1 – Beginning                     |
|---------------------------|-----------|-----------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------------|-----------------------------------------|
| Concept Identification    | 25%       | Identifies all key concepts from literature accurately and comprehensively  | Identifies most key concepts with minor omissions | Identifies limited concepts; some are irrelevant or missing | Fails to identify essential concepts    |
| Linkages                  | 25%       | Establishes clear, meaningful, and accurate relationships between concepts  | Shows correct relationships with minor gaps       | Relationships are weak, unclear, or partially incorrect     | No meaningful relationships established |
| Organization              | 20%       | Concepts are well-structured hierarchically with logical flow and grouping  | Mostly organized with minor structural issues     | Some organization but lacks clear hierarchy                 | No logical organization or structure    |
| Integration of Literature | 20%       | Integrates multiple studies effectively to show connections and comparisons | Shows some integration of literature              | Limited integration; mostly isolated concepts               | No integration of literature evidence   |
| Clarity                   | 10%       | Concept map is clear, neat, visually structured, and easy to interpret      | Generally clear with minor readability issues     | Clarity is reduced; difficult to interpret in parts         | Poorly presented and hard to understand |

# QUESTION 1

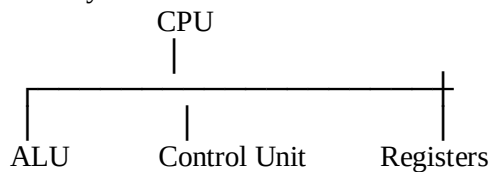
## COMPUTER SYSTEM

The Computer System is an electronic system that accepts input, processes data, stores information, and produces meaningful output. It consists of four major components:



## CPU (Central Processing Unit)

The CPU is the brain of the computer. It performs all processing operations by executing program instructions stored in memory.

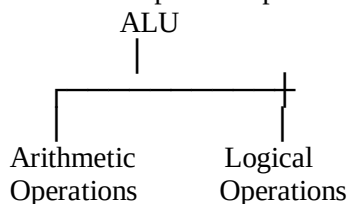


### 1. Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic and logical operations.

Functions

- Addition
- Subtraction
- Multiplication
- Division
- AND
- OR
- XOR
- NOT
- Comparison operations

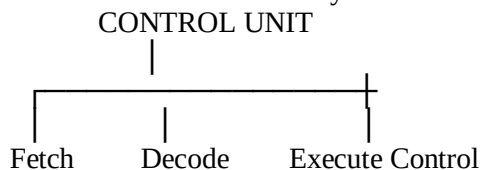


### 2. Control Unit (CU)

The Control Unit manages and coordinates all computer operations.

Functions

- Fetches instructions from memory
- Decodes instructions
- Sends control signals
- Controls ALU operations
- Coordinates memory and I/O devices

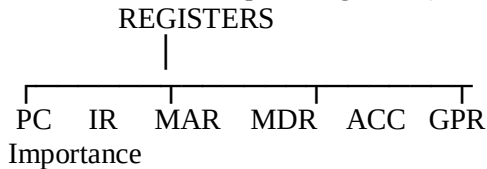


### 3. Registers

Registers are high-speed storage locations inside the CPU.

Types

- Program Counter (PC)
- Instruction Register (IR)
- Memory Address Register (MAR)
- Memory Data Register (MDR)
- Accumulator (ACC)
- General Purpose Registers (GPR)

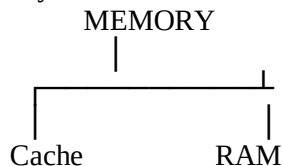


Importance

- Temporary data storage
- Fast instruction execution
- Reduces memory access time

### MEMORY

Memory stores both instructions and data.

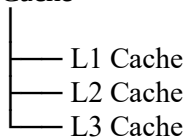


#### Cache Memory

Cache is a small but very fast memory located close to the CPU.

Levels

Cache



Advantages

- Faster access
- Reduces CPU waiting time
- Improves overall performance

#### RAM (Random Access Memory)

RAM is the main memory used during program execution.

Types

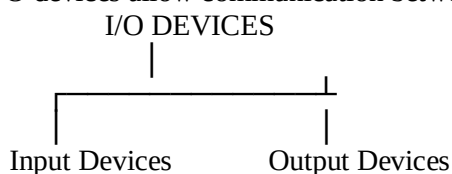
- SRAM
- DRAM

Characteristics

- Temporary storage
- Volatile memory
- Larger capacity than cache
- Stores active programs and data

### INPUT / OUTPUT DEVICES

I/O devices allow communication between the user and the computer.



## Input Devices

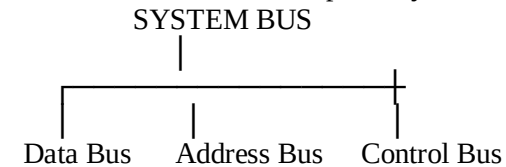
- Keyboard
- Mouse
- Scanner
- Microphone
- Webcam

## Output Devices

- Monitor
- Printer
- Speaker
- Projector

## BUS STRUCTURE

A Bus is a communication pathway connecting the CPU, memory, and I/O devices.



### Data Bus

- Transfers data
- Bidirectional

### Address Bus

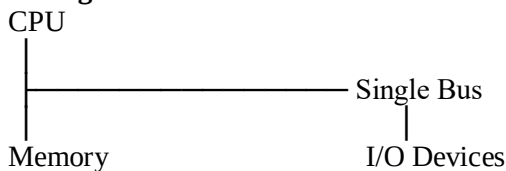
- Transfers memory addresses
- Unidirectional

### Control Bus

- Transfers control signals
- Read/Write operations
- Interrupt signals

## TYPES OF BUS STRUCTURES

### 1. Single Bus Structure



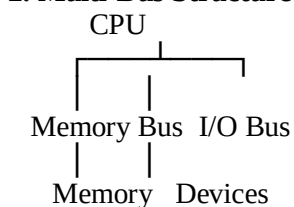
#### Advantages

- Simple design
- Low cost

#### Disadvantages

- Bus contention
- Slow performance
- One transfer at a time

### 2. Multi-Bus Structure



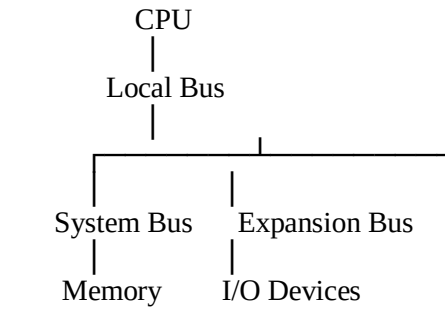
#### Advantages

- Parallel communication
- Faster execution
- Reduced congestion

#### Disadvantages

- Higher hardware cost
- More complex design

### 3. Hierarchical Bus Structure

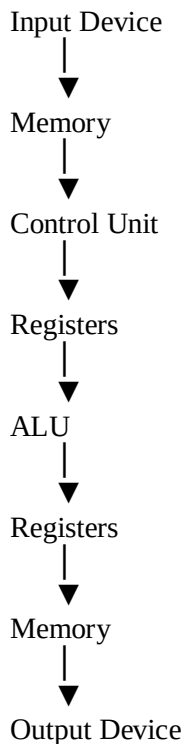


#### Advantages

- High scalability
- Efficient communication
- Better performance
- Reduced bottlenecks

---

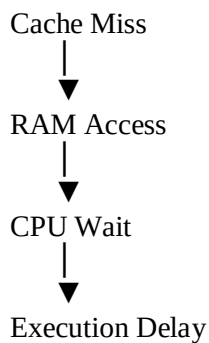
### DATA FLOW DURING EXECUTION



### BOTTLENECKS

Performance may decrease because of:

- Slow RAM access
- Cache miss
- Bus congestion
- I/O waiting time
- CPU idle cycles
- Limited bus bandwidth



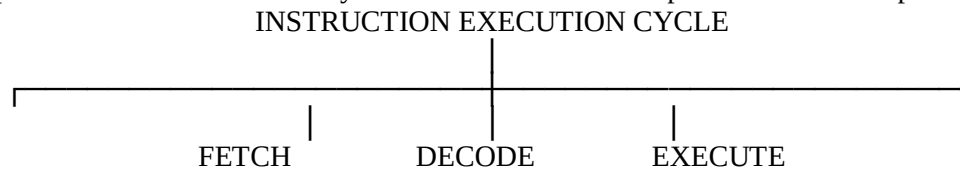
## PERFORMANCE IMPROVEMENTS

- Use L1/L2/L3 cache
- Increase RAM speed
- Use Multi-Bus architecture
- Implement Hierarchical Bus
- Faster processors
- Parallel data transfer
- Efficient bus arbitration

## QUESTION 2

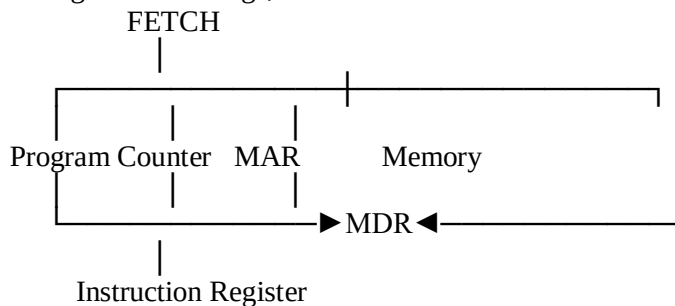
### INSTRUCTION EXECUTION CYCLE

The Instruction Execution Cycle is the sequence of operations performed by the CPU to execute every instruction stored in memory. Every instruction passes through three major stages: Fetch, Decode, and Execute. These stages work together to process instructions efficiently and determine the overall performance of the processor.



### FETCH STAGE

During the Fetch stage, the CPU retrieves the instruction from the main memory.



### Components Used

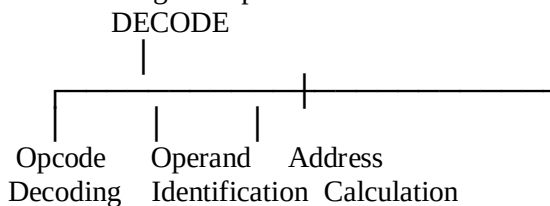
- Program Counter (PC)
- Memory Address Register (MAR)
- Memory Data Register (MDR)
- Instruction Register (IR)

### Functions

- Reads the next instruction.
- Updates the Program Counter.
- Transfers instruction into the Instruction Register.
- Begins the execution cycle.

### DECODE STAGE

The Decode stage interprets the instruction fetched from memory.

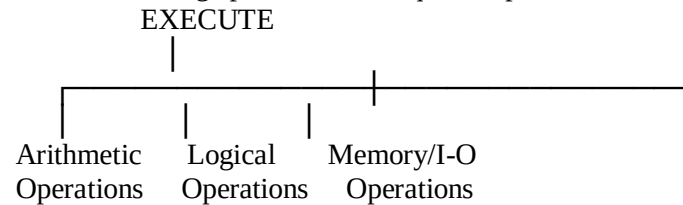


### Functions

- Decodes machine instruction.
- Identifies operation.
- Determines operands.
- Generates control signals.
- Selects ALU operation.

## EXECUTE STAGE

The Execute stage performs the required operation.

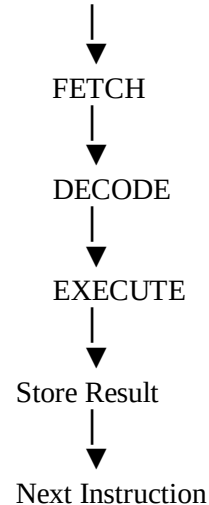


### Functions

- Performs arithmetic operations.
- Executes logical operations.
- Stores results.
- Updates registers.
- Writes results back to memory if required.

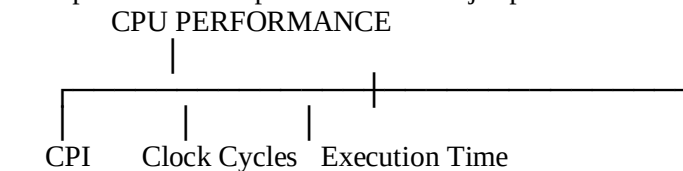
## COMPLETE EXECUTION FLOW

Instruction in Memory



## CPU PERFORMANCE PARAMETERS

CPU performance depends on three major parameters.



### CPI (Cycles Per Instruction)

CPI represents the average number of clock cycles required to execute one instruction.

#### Low CPI

✓ Faster execution

#### High CPI

✗ Slower execution

### Clock Cycle

A Clock Cycle is one pulse generated by the processor clock.

Higher Clock Frequency → More Clock Cycles per Second → Faster Processing

### Execution Time

Execution Time is the total time required to execute a program.

Execution Time depends on:

- Number of Instructions
- CPI
- Clock Cycle Time

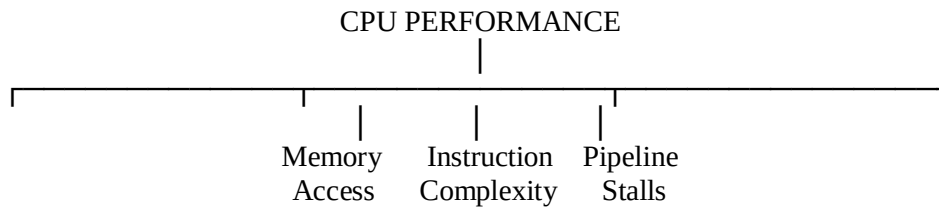
Execution Time





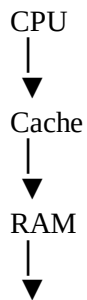
▼  
 Instruction Count  
 ×  
 CPI  
 ×  
 Clock Cycle Time

## FACTORS AFFECTING CPU PERFORMANCE



## MEMORY ACCESS

Memory Access Time significantly affects execution speed.



Hard Disk (if needed)

### Faster Memory Access

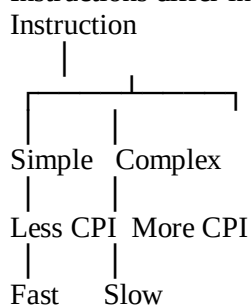
- L1 Cache
- L2 Cache
- L3 Cache

### Slower Memory Access

- Cache Miss
- RAM Delay
- Secondary Storage Access

## INSTRUCTION COMPLEXITY

Instructions differ in complexity.



Examples:

Simple Instructions

- MOV
- ADD

Complex Instructions

- DIV
- Floating Point Operations
- String Manipulation

## PIPELINE

Pipeline improves processor performance by executing multiple instructions simultaneously.

Instruction 1

Fetch → Decode → Execute

Instruction 2

Fetch → Decode → Execute

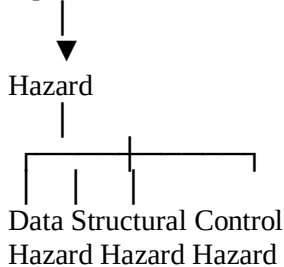
Instruction 3

Fetch → Decode → Execute

### PIPELINE STALLS

Pipeline stalls occur when instruction execution is interrupted.

Pipeline



#### Data Hazard

Instruction depends on previous instruction.

#### Structural Hazard

Hardware resource conflict.

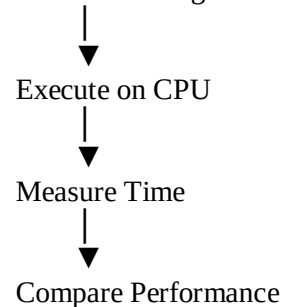
#### Control Hazard

Occurs due to branch instructions.

### BENCHMARKING

Benchmarking measures CPU performance using standard programs.

Benchmark Programs



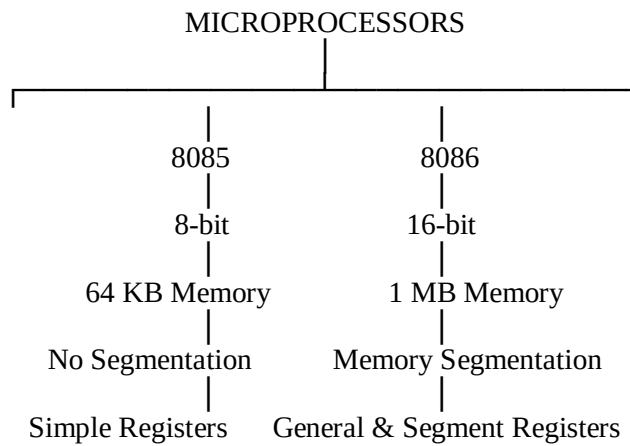
### PERFORMANCE IMPROVEMENTS

- Increase clock frequency.
- Reduce CPI.
- Improve cache memory.
- Use pipelining.
- Reduce pipeline hazards.
- Optimize instruction scheduling.
- Improve branch prediction.
- Increase parallel execution.

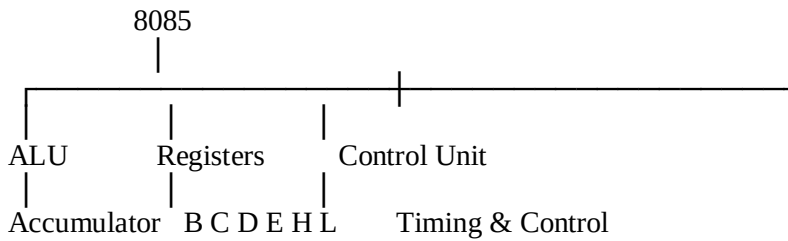
## QUESTION 3

### COMPARISON OF 8085 AND 8086 ARCHITECTURES

The 8085 and 8086 are microprocessors developed by Intel. The 8085 is an 8-bit microprocessor designed for basic computing applications, whereas the 8086 is a 16-bit microprocessor with advanced features such as memory segmentation, pipelining, and multitasking support. The architectural improvements in the 8086 result in faster execution, better memory management, and higher efficiency compared to the 8085.



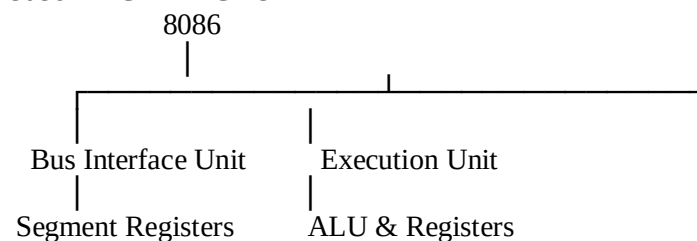
## 8085 ARCHITECTURE



### Features of 8085

- 8-bit data bus
- 16-bit address bus
- Maximum memory addressing of **64 KB**
- Six general-purpose registers (B, C, D, E, H, L)
- No memory segmentation
- Limited instruction set
- Supports basic arithmetic, logical, data transfer, and branching instructions
- Suitable for simple embedded and control applications

## 8086 ARCHITECTURE



### Features of 8086

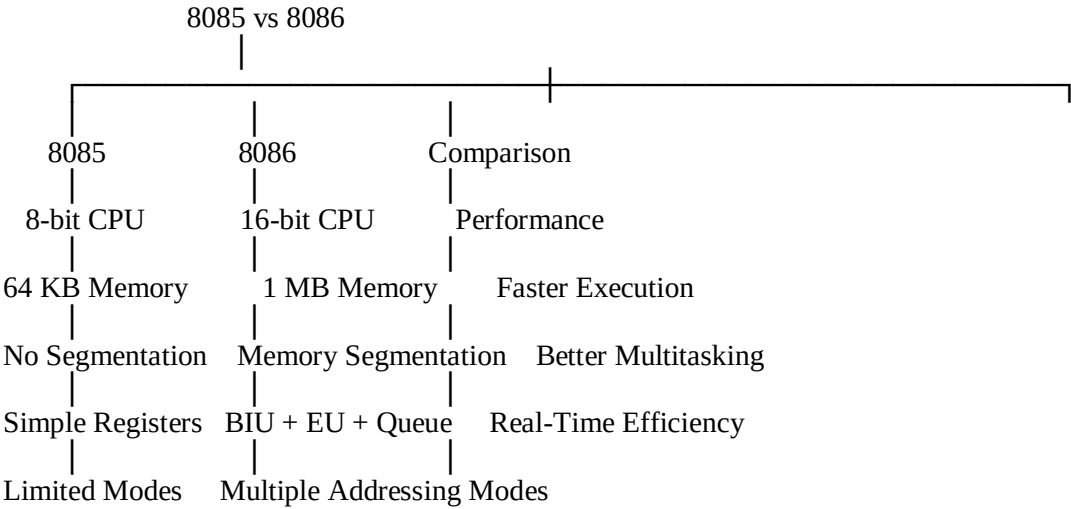
- 16-bit data bus
- 20-bit address bus
- Supports **1 MB memory**
- Memory segmentation using **Code, Data, Stack, and Extra segments**
- Contains BIU (Bus Interface Unit) and EU (Execution Unit)
- Instruction queue enables pipelined execution
- Larger instruction set with multiple addressing modes
- Faster processing and better multitasking capability

## COMPARISON TABLE

| Feature        | 8085   | 8086   |
|----------------|--------|--------|
| Processor      | 8-bit  | 16-bit |
| Address Bus    | 16-bit | 20-bit |
| Maximum Memory | 64 KB  | 1 MB   |

| Feature                                                                                        | 8085            | 8086                        |
|------------------------------------------------------------------------------------------------|-----------------|-----------------------------|
| Registers                                                                                      | Basic Registers | General + Segment Registers |
| Memory Segmentation                                                                            | Not Available   | Available                   |
| Instruction Queue                                                                              | No              | Yes (6-byte queue)          |
| Addressing Modes                                                                               | Limited         | Multiple                    |
| Processing Speed                                                                               | Lower           | Higher                      |
| Multitasking                                                                                   | Not Supported   | Better Support              |
| Real-Time Applications Small Embedded Systems Industrial Automation, Robotics, Control Systems |                 |                             |

CONCEPT MAP



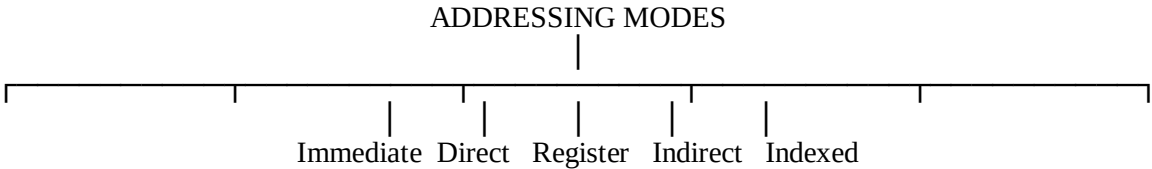
EFFECT ON PERFORMANCE

- **8085** provides simple program execution and is suitable for small embedded applications where memory and processing requirements are limited.
- **8086** improves execution speed through **instruction prefetching**, **memory segmentation**, and **multiple addressing modes**.
- The **Bus Interface Unit (BIU)** and **Execution Unit (EU)** work simultaneously, reducing CPU idle time and increasing throughput.
- Memory segmentation allows efficient organization of larger programs, while the instruction queue supports faster execution.
- These architectural enhancements make the **8086 more suitable for multitasking and real-time applications** than the 8085.

QUESTION 4

ADDRESSING MODES AND INSTRUCTION TYPES

Addressing Modes specify how the CPU locates the operand required for executing an instruction. Different addressing modes influence memory access time, instruction size, execution speed, and overall processor performance. The choice of addressing mode depends on the application, data location, and instruction type.



### Immediate Addressing Mode

- Operand is stored within the instruction itself.
- No additional memory access is required.
- Fastest execution with small instruction size.
- **Example:** MOV A, 25H

### Direct Addressing Mode

- Memory address of the operand is specified in the instruction.
- Requires one memory access.
- Suitable for accessing fixed memory locations.
- **Example:** LDA 2500H

### Register Addressing Mode

- Operand is stored in CPU registers.
- Very high execution speed due to register access.
- Commonly used for arithmetic and logical operations.
- **Example:** MOV A, B

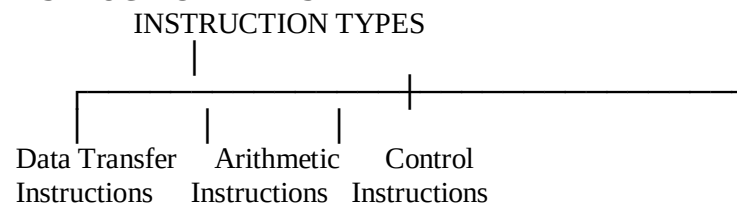
### Indirect Addressing Mode

- Instruction contains the address of a register that points to the memory location.
- Flexible for dynamic data structures.
- Requires additional memory reference.
- **Example:** MOV A, M

### Indexed Addressing Mode

- Effective address is calculated using a base address and an index register.
- Efficient for arrays, tables, and looping operations.
- Widely used in modern processors.

## INSTRUCTION TYPES



### Data Transfer Instructions

- MOV
- LOAD
- STORE
- PUSH
- POP

**Purpose:** Transfer data between registers, memory, and I/O devices.

### Arithmetic Instructions

- ADD
- SUB
- MUL
- DIV
- INC
- DEC

**Purpose:** Perform mathematical operations using operands.

### Control Instructions

- JMP
- CALL
- RET
- LOOP
- JNZ

**Purpose:** Control the execution sequence of a program.

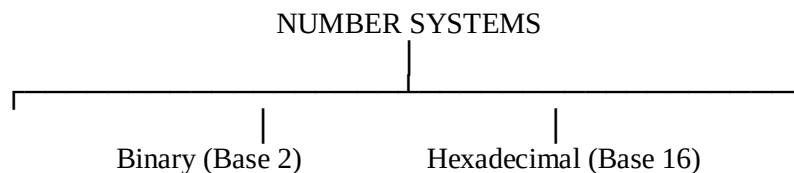
## EFFECT ON PERFORMANCE

| Addressing Mode | Memory Access                | Instruction Size | Execution Speed | Common Applications                 |
|-----------------|------------------------------|------------------|-----------------|-------------------------------------|
| Immediate       | No memory access             | Small            | Very Fast       | Constants and initialization        |
| Register        | Register access only         | Small            | Fastest         | Arithmetic and logical operations   |
| Direct          | One memory access            | Medium           | Moderate        | Fixed memory locations              |
| Indirect        | Multiple memory references   | Medium           | Slower          | Pointers and linked data structures |
| Indexed         | Address calculation required | Larger           | Moderate        | Arrays, matrices, loops             |

## QUESTION 5

### NUMBER SYSTEMS, INSTRUCTION REPRESENTATION, MEMORY STORAGE & DEBUGGING

A Number System is a method of representing data in computers. Computers internally use the Binary Number System (0 and 1) because electronic circuits operate using two voltage levels (ON/OFF). For easier understanding by programmers, Hexadecimal (Base 16) is used to represent binary values in a compact form. These number systems play a vital role in instruction representation, memory organization, debugging, and low-level programming.



#### Binary Number System

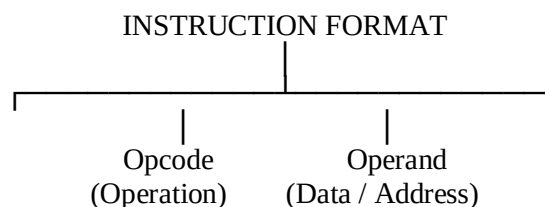
- Uses only 0 and 1.
- Fundamental language of computers.
- Each binary digit is called a bit.
- Used to represent instructions, addresses, and data in memory.
- Example: 10101100
- 

#### Hexadecimal Number System

- Uses digits 0–9 and letters A–F.
- One hexadecimal digit represents 4 binary bits.
- Easier to read and write than long binary numbers.
- Example:  $AC_{16} = 10101100_2$

## INSTRUCTION REPRESENTATION

Machine instructions are represented in binary form and stored in memory. Every instruction contains an Opcode (operation code) and an Operand (data or memory address).

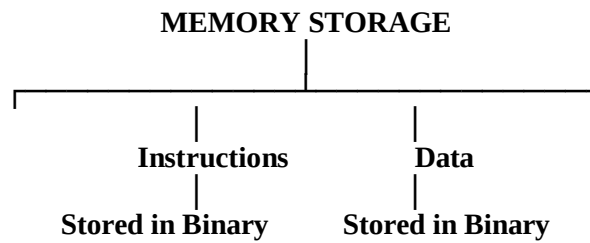


#### Examples

- ADD
- SUB
- MOV
- LOAD
- STORE

The CPU fetches the instruction, decodes the opcode, executes the operation, and stores the result.

## MEMORY STORAGE

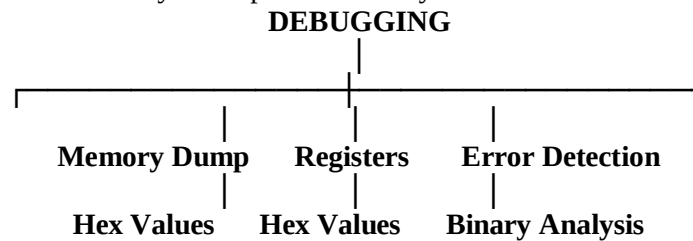


### Memory Characteristics

- Memory stores instructions and data in binary format.
- Memory addresses are commonly displayed in hexadecimal.
- The CPU accesses memory using address buses and data buses.

## DEBUGGING PROCESS

Debugging is the process of identifying and correcting errors in programs. During debugging, hexadecimal notation is preferred because it is compact and directly corresponds to binary values.



### Advantages of Hexadecimal in Debugging

- Simplifies memory addresses.
- Makes machine code easier to read.
- Reduces errors when analyzing binary data.
- Widely used in assemblers, debuggers, and embedded systems.

## Conclusion

Binary and hexadecimal number systems are essential for computer architecture. **Binary** is used internally by the CPU to process instructions and store data, while **hexadecimal** provides a compact and readable representation of binary values. Together, they simplify instruction representation, memory addressing, debugging, and low-level programming, making computer systems more efficient and easier to understand.